

Welcome back

How was lab? Was anything confusing?

Notes for week 1: <https://tinyurl.com/4ypzsmzu>

Important note: do not read ahead

```
if <condition>:
    <expression>
elif <condition>:
    <expression>
else:
    <expression>

while <condition>:
    <expression>

# list, string, dictionary, or range
for <iterable expression>:
    <expressions>
```

Review

Last time, we covered:

- For loop
- Basic functions

While loop syntax

The syntax for while loops is:

```
while <condition>:
    <expression>
```

As with `if` statements, the `<condition>` can be anything that evaluates to a boolean. The `<expression>` can be complex.

```

num = 0
while num < 4:
    print(num)
    num += 1

print("Done!")

a = 1
b = 1
num = 0

while num < 8:
    print(a)
    c = a + b
    a = b
    b = c
    num += 1
    # print(a, b, c, num)
    # print()

```

Basic syntax for `for` loops

The basic syntax for `for` loops is

```

for <variable> in <iterable expression>:
    <expression> / <body>

```

Don't worry too much about what an `<iterable expression>` is for the time being. For this lecture, it will always be a list, string, dictionary, or `range`. We will cover `range` later.

As with `ifs` and `whiles`, the `<expression>` can be complex.

```

for x in [10, 9]:
    print(x)
print()

y = [10, 9]
index = 0
while index < len(y):
    x = y[index]
    print(x)
    index += 1
print()

x = [10, 9][0] # x = 10
print(x)

```

```
x = [10, 9][1] # x = 9
print(x)
```

```
10
9
```

```
10
9
```

```
10
9
```

```
x = [1, 2, 3, 4, 5, 6]
for y in x:
    print(y)
print()
```

```
y = x[0]
print(y)
```

```
y = x[1]
print(y)
```

```
y = x[2]
print(y)
```

```
y = x[3]
print(y)
```

```
y = x[4]
print(y)
```

```
y = x[5]
print(y)
```

```
1
2
3
4
5
6
```

```
1
2
3
4
5
6
```

```
x = [4, 3, 8]
sum_of_squares = x[0] * x[0] + x[1] * x[1] + x[2] * x[2]
```

```

print(sum_of_squares)

sum_of_squares = 0

for y in x:
    sum_of_squares += y * y # 25 + 64 = 89
print(sum_of_squares)

```

```

89
89

```

```

y = ""
z = "hello"
for s in z: # s = z[1] = "e"
    # y = "h"
    y += s # s = "e"
    # y = "h" + "e" = "he"

```

```

print(y)

```

```

hello

```

```

x = {
    "key": "value",
    4: 10,
}

```

```

for key in x: # key = 4
    print(x[key]) # x[4] -> 10

```

```

value
10

```

```

print(1, 4)

```

```

1 4

```

```

for x in [1, 2, 3]: # x = 2
    for y in [4, 5]: # y = 4
        print(x, y) # x = x, y = 4

```

```

print("Done!")

```

```

1 4
1 5
2 4
2 5
3 4
3 5
Done!

```

```

for x in range(3): # [0, 1, 2]
    print(x)
print()

for x in [0, 1, 2]:
    print(x)

0
1
2

0
1
2

# x = [0, 1, 2, 3, 4, ....]
# x[0:3] -> [x[0], x[1], x[2]] -> [0, 1, 2]
# x[3:5] -> [x[3], x[4]] -> [3, 4]
# x[3:8:2] -> [x[3], x[5], x[7]] -> [3, 5, 7]

list(range(3)) # [0, 1, 2]
list(range(3, 8)) # [3, 4, 5, 6, 7]
list(range(3, 8, 2)) # [3, 5, 7]

# [0, 2, 4, 6]
list(range(0, 7, 2))

# [0, 2, 4, ..., 100]
n = 101
list(range(0, n, 2))

# [1, 3, 5, ..., 99]
n = 100
list(range(1, n, 2))

lst = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]

for x in range(3, 10, 3):
    print(x)
print()

for x in lst[3:10:3]:
    print(x)
print()

start = 3
end = 10
step = 3

index = start

```

```

while index < end:
    x = lst[index]
    print(x)
    index += step
print()

```

```

3
6
9

```

```

3
6
9

```

```

3
6
9

```

```

for x in [1, 2, 3]:
    y = 0
    while y < x:
        print(y)
        y += 1
    print()

```

```

0

```

```

0
1

```

```

0
1
2

```

```

for x in range(5):
    print(x)

```

```

x = [4, 3, 2, 1]

```

```

# len(x) = 4
# list(range(len(x)))
# list(range(4))
# [0, 1, 2, 3]

```

```

for ind in range(len(x)): # ind = 0
    print(ind, x[ind])    # ind = 0, x[0] = 4

```

```

0 4
1 3

```

```
2 2
3 1
```

```
y = [5, 6, 7, 8]
y[4]
```

```
-----
-----
IndexError                                Traceback (most recent call
last)
```

```
Cell In[60], line 2
      1 y = [5, 6, 7, 8]
----> 2 y[4]
```

```
IndexError: list index out of range
```

```
x = [1, 2, 3, 4]
y = [5, 6, 7, 8]
```

```
# len(x) = 4
```

```
mul_sum = 0
# range(len(x)) -> [0, 1, 2, 3]
for ind in range(len(x)): # ind = 4
    # x[ind] = 4
    # y[ind] = 8
    # mul_sum = 38
    mul_sum += x[ind] * y[ind]
    # mul_sum = 38 + 32 = 70
```

```
mul_sum
```

```
70
```

```
x = [1, 4, 7, 9, 11, 15]
```

```
# Compute the squared sum of the values
# of x until we reach 7
```

```
squared_sum = 0
for y in x: # y = x[2] = 7
    if y == 7: # (7 == 7) = True
        break
    # squared_sum = 1 + 16 = 17
    squared_sum += y * y
```

```
squared_sum
```

```
# Breaks can come anywhere in the
# body of a for loop
```

```
for x in [1, 2, 3]: # x = 1
```

```

    print(x)
    break
print()

lst = [1, 2, 3]
index = 0

while index < len(lst):
    x = lst[index]
    print(x)
    break
    index += 1
print()

print("Done!")

1

1

Done!

lst = [1, 4, 7, 9, 11, 12, 15]

# Goal: print only the even numbers in a list
for x in lst: # x = 1
    if x % 2 == 0:
        print(x)
print()

for x in lst: # x = 4
    if x % 2 != 0:
        continue
    print(x)

4
12

4
12

x = [1, 4, 7, 9, 11, 15]

# Compute the squared sum of
# the _even_ values of x
squared_sum = 0
for y in x: # y = x[1] = 4
    if y % 2 == 1: # False
        continue
    # squared_sum = 0 + 4 * 4 = 16
    squared_sum += y * y

```

squared_sum

Review redux: syntax for functions

The general syntax for functions is:

```
def function_name(<arguments>):  
    <body>  
    return <expression>
```

There are several things to note about the syntax (we will go through examples):

- You can have zero, one, or more arguments.
- You need the `:` after the function
- The body can be complex!
- You technically don't need a return statement at the end, but we will almost always have a return at the end

```
def add_two(x, y):  
    # x = 1  
    # y = 2  
    s = x + y # s = 3  
    return s  # s = 3  
  
add_two(1, 2)  
3  
  
add_two(4, 5)  
9  
  
def fahrenheit_to_celsius(f):  
    # ???  
    return 0  
  
def celsius_to_fahrenheit(f):  
    # ???  
    return 0  
  
def add_two(x, y): # x = 6, y = 3  
    # add x and y  
    z = x * y # z = 6 * 3 = 18  
    z += x    # z = z + 6 = 18 + 6 = 24  
    z /= y    # z = z / y = 24 / 3 = 8.0  
    return x + y # 9  
  
# Calling a function
```

```

a = add_two(6, 3) # 9
a

def mystery(x, y):
    return x + y
    x = 1
    y = 2
    return x + y

a = mystery(6, 3) # 9
a

def mystery(x, y):
    if x % 2 == 0:
        return x + y
    else:
        return x - y

mystery(7, 4)

int("1")

```

Using other concepts in functions

We can use other concepts we learned within functions, including `ifs`, `whiles`, and `for`s.

```

def minimum_two(x, y):
    # x = 1
    # y = 2
    if x < y:
        return x
    else:
        return y

minimum_two(1, 2)

1

def min_list(l):
    # l = [57, 391, 21, 458]
    minimum = l[0] # minimum = 57
    for x in l[1:len(l)]: # x = l[1] = 391
        minimum = minimum_two(minimum, x)
        # minimum = 57
    return minimum

min_list([57, 391, 21, 458])

```

```
def contains_pineapple(dictionary):
    if 'pineapple' in dictionary:
        return True
    else:
        return False

x = {
    'apple': 1,
    'cherry': 2,
    'pineapple': 3,
}
contains_pineapple(x)
contains_pineapple(x)
```

Reusing functions

Once we define a function once, we can use it anywhere. We can:

- Use functions within expressions
- Call functions within functions

```
if 'apple' in x:
    y = contains_pineapple(x)
else:
    y = not contains_pineapple(x)

y

def minimum_two(x, y):
    if x < y:
        return x
    else:
        return y

minimum_two(1, 2)

1

# minimum_two(1, 2)
minimum_two(1, minimum_two(2, 3))

def minimum_three(x, y, z):
    if x < y:
        if x < z:
            return x
        else:
            return z
    else: # x >= y
```

```

        if y < z:
            return y
        else:
            return z

minimum_three(1, 2, 3)
1

def minimum_three(x, y, z):
    # x = 1
    # y = 2
    # z = 3
    min_xy = minimum_two(x, y) # = 1
    min_xyz = minimum_two(min_xy, z) # = 1
    return min_xyz

minimum_three(1, 2, 3)
1

```

Variable scope

So far (up until functions), we've used variables that can be used anywhere in Jupyter. With functions, we will start to use variables that can only be used in certain places.

Where variables can be used is called *variable scope*.

```

def variable_scope_example():
    scoped_variable = 0
    return scoped_variable

def variable_scope_example2():
    scoped_variable2 = 0
    print(scoped_variable)
    return scoped_variable

print(variable_scope_example())
print(variable_scope_example2())

# scoped_variable

0

```

```

-----
-----
NameError                                Traceback (most recent call
last)
Cell In[116], line 11

```

```

    8     return scoped_variable
    10 print(variable_scope_example())
--> 11 print(variable_scope_example2())

Cell In[116], line 7, in variable_scope_example2()
    5 def variable_scope_example2():
    6     scoped_variable2 = 0
--> 7     print(scoped_variable)
    8     return scoped_variable

NameError: name 'scoped_variable' is not defined

# This is called using a global variable.
# Avoid using global variables.

x = 0

def variable_scope_example_two():
    print(x)

variable_scope_example_two()

0

```

Built in functions

Python provides us *built in functions*, which are functions that we can use for free!

We've already seen some examples: `type`, `int`, etc.

```

type(1)
int
int(True)
1
min(1, 2)
1
max(1, 2)
2
lst = [5, 10, 6, 2, 4, 9]
lst[1:3]

```

```
[10, 6]

def slice_two(slice_lst, start, end):
    return_lst = []

    index = start
    while index < end:
        return_lst.append(slice_lst[index])
        index += 1
    return return_lst

slice_two(lst, 1, 3) # lst[1:3]

slice_one_end(lst, 3) # lst[:3]
slice_one_start(lst, 3) # lst[3:]

[10, 6]
```